

TrieHard: Efficient Prefix Search Using Trie (Prefix Tree)

Eishal Keshwani (ek10843)
Muhammad Hadi (mh10955)
Syed Ebad Ali (sa10805)

Data Structures & Algorithms Course Project

Problem Statement & Significance

Modern applications such as search engines, mobile keyboards, and spell-checking systems require fast word suggestions while the user types. Traditional search methods scan the entire dataset and are inefficient for real-time suggestions.

This project implements an Auto-Complete System using a Trie (Prefix Tree) data structure that efficiently stores and retrieves words based on prefixes. The system provides real-time suggestions, supports word insertion and deletion, and ranks suggestions based on usage frequency.

The significance of this project lies in demonstrating how Trie structures improve prefix-based search performance compared to linear search methods.

Data Structure & Algorithm Justification

A Trie (Prefix Tree) is a tree-based data structure used for efficient storage and retrieval of strings. Each node represents a character and words sharing common prefixes follow the same path.

The Trie was selected because:

- Prefix search complexity depends only on word length, not dataset size
- It supports fast insertion, deletion, and lookup
- It enables efficient real-time autocomplete suggestions
- It is widely used in search engines and keyboard prediction systems

The project implements the following operations using Trie:

- Word insertion
- Word deletion
- Exact word search
- Prefix-based search
- Autocomplete suggestion generation

System Implementation Overview

The system consists of three modules:

- **main.py**: Implements Trie structure and algorithms
- **DSA_FEATURES.py**: Handles persistence, ranking, and history tracking
- **DSA_GUI.py**: Provides graphical user interface using CustomTkinter

The GUI displays suggestions dynamically as the user types and allows adding, deleting, and selecting words interactively.

Time Complexity Analysis

Let:

L = length of word, P = length of prefix, N = number of stored words

The time complexity of main Trie operations is as follows:

Trie Creation

Function: `trie()`

$$O(1)$$

Creates an empty dictionary representing the root node.

Node Creation

Function: `node()`

$$O(1)$$

Creates a single node with end-of-word flag.

Insertion Operation

Function: `insert(trie, word)`

$$O(L)$$

Traverses each character once and inserts nodes if missing.

Deletion Operation

Function: `delete(trie, word)`

$$O(L)$$

Recursively traverses the Trie and removes unused nodes safely.

Update Operation

Function: `update(trie, word)`

$$O(L)$$

Calls insertion internally.

Exact Word Search

Function: `isWord(trie, word)`

$$O(L)$$

Traverses characters sequentially and checks end-of-word marker.

Prefix Search Operation

Function: `allWordsWithPrefix(trie, prefix)`

$$O(P + K)$$

Where:

$$K = \text{number of characterseexploredinsubtree}$$

The function first traverses the prefix path and then explores subtree recursively.

Memory Complexity Analysis

Trie memory usage depends on number of stored characters.

Worst case:

$$O(N \times L)$$

However, memory usage improves because common prefixes share nodes.

Example:

hello, hell, help

share the same initial nodes.

The project also optimizes memory by removing unused nodes during deletion.

Conclusion

The project successfully demonstrates the implementation of a Trie-based autocomplete system supporting insertion, deletion, search, and prefix suggestions in real time.

Compared to traditional search methods, the Trie structure significantly improves prefix search efficiency and enables scalable suggestion systems suitable for real-world applications such as keyboards and search engines.

The system further enhances usability by ranking suggestions based on frequency and storing persistent word data using CSV storage.